Arrays are Covariant

Definition (2 Lines)

Arrays in Java are covariant, which means if `Child` extends `Parent`, then `Child[]` is also considered a subtype of `Parent[]`.

This allows assigning a child array to a parent array reference, but the actual array type **does not change**.

Diagram

```
class Animal
      ▲
      |
class Dog
```

Since `Dog` extends `Animal`

```
Dog[]  ──────────►  Animal[]
        (Allowed)
```

Memory Diagram

```
Dog[] dogs = new Dog[2];
Animal[] animals = dogs;
```

```
dogs ──────┐
           |
           ▼
      +-------------+
      | Dog | Dog   |   ← Actual Object = Dog[]
      +-------------+
           ▲
           |
animals ───┘
```

Important: `animals` is only another reference. The actual object is still a `Dog[]`.

Safe Operation ✅

```
animals[0] = new Dog();
```

```
Dog[]  ← Dog ✔
```

Unsafe Operation ❌

```
animals[1] = new Cat();
```

```
Dog[]  ← Cat ❌
```

```
Runtime:
ArrayStoreException
```

Why?

The compiler checks the **reference type** (`Animal[]`), so the code compiles.

The JVM checks the **actual array type** (`Dog[]`), so storing a `Cat` is rejected at runtime.

One-Line Interview Answer

> **Changing the reference type does not change the actual array object. The JVM remembers the real component type (`Dog[]`) and throws `ArrayStoreException` if an incompatible object is stored.**